Task 04 – NumPy & Pandas for Data Analysis

Report

Name: Abdul Rehman
Task: 04
Course: AI & Software Development
Date: 08/07/2026

Part A - Task 1: NumPy Arrays

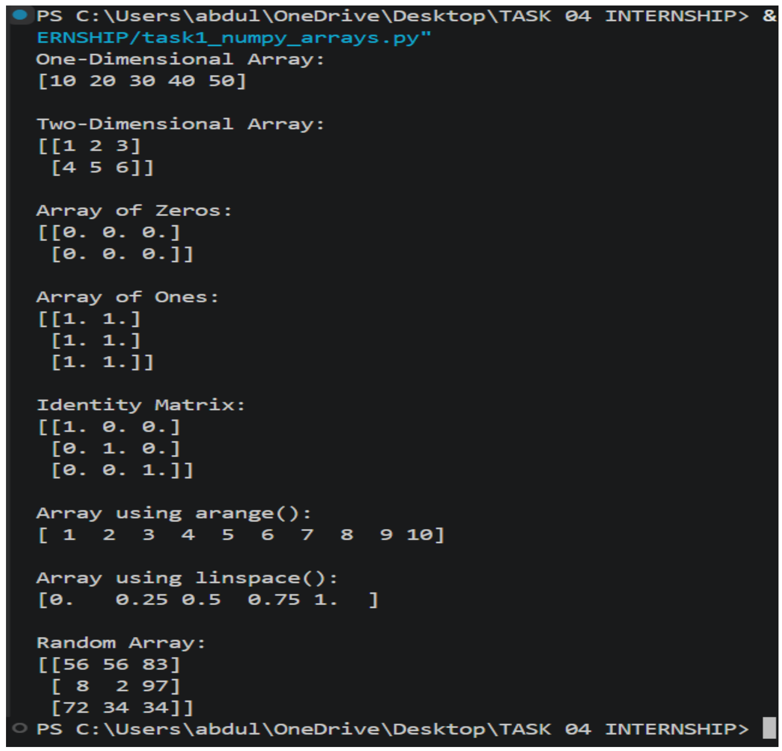
Aim

Create one-dimensional and multi-dimensional NumPy arrays using different methods.

Python File

task1_numpy_arrays.py

Output Screenshot



Conclusion

I created NumPy arrays using different functions such as array(), zeros(), ones(), eye(), arange(), linspace(), and random integers.

Part A - Task 2: Indexing, Slicing, Reshaping and Mathematical Operations

Aim

Perform basic NumPy array operations.

Python File

task2_numpy_operations.py

Output Screenshot

```
PS C:\Users\abdul\OneDrive\Desktop\TASK
ERNSHIP/task2_numpy_operations.py"
Original Array:
[10 20 30 40 50 60]

First Element: 10
Last Element: 60

Elements from index 1 to 4:
[20 30 40 50]

2D Array:
[[1 2 3]
 [4 5 6]]

Element at Row 1, Column 2: 2

Original Array:
[ 1  2  3  4  5  6  7  8  9 10 11 12]

Reshaped Array (3x4):
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]

Array A: [1 2 3]
Array B: [4 5 6]

Addition:
[5 7 9]

Subtraction:
[-3 -3 -3]

Multiplication:
[ 4 10 18]
```

Conclusion

I learned how to access, reshape, slice, and perform mathematical operations on arrays.

Part A - Task 3: Broadcasting

Aim

Demonstrate broadcasting in NumPy.

Python File

task3_broadcasting.py

Output Screenshot

```
PS C:\Users\abdul\OneDrive\Desktop\TASK 04 INTERNSHIP>
ERNSHIP/task3_broadcasting.py"
[10 20 30]

Broadcasting Result (Matrix + Vector):
[[11 22 33]
 [14 25 36]]

Multiply Matrix by 2:
[[ 2  4  6]
 [ 8 10 12]]
```

Conclusion

Broadcasting allows operations on arrays of different shapes without writing loops.

Advantages of Broadcasting

- Faster execution
- Less code
- Better memory usage
- Easy to read

Part A - Task 4: Vectorization

Aim

Compare vectorized operations with traditional loops.

Details:

Vectorization performs operations on the entire array at once, whereas traditional loops process elements one by one. Vectorized operations are generally much faster.

Python File

task4_vectorization.py

Output Screenshot

```
PS C:\Users\abdul\OneDrive\Desktop\TASK 04 INTERNSHIP> & D:\Python\python.exe "c:/Users/abdul/OneDrive/Desktop/TASK 04 INT
ERNSHIP/task4_vectorization.py"
Loop Time:
0.26305508613586426 seconds

Vectorized Operation Time:
0.004441499710083008 seconds

First 10 Elements (Loop):
[np.int64(2), np.int64(4), np.int64(6), np.int64(8), np.int64(10), np.int64(12), np.int64(14), np.int64(16), np.int64(18),
 np.int64(20)]

First 10 Elements (Vectorized):
[ 2  4  6  8 10 12 14 16 18 20]
```

Conclusion

Vectorized operations are faster and require less code than loops.

Part A - Task 5: Linear Algebra

Aim

Perform dot product, matrix multiplication, transpose, and inverse.

Python File

task5_linear_algebra.py

Output Screenshot

```
PS C:\Users\abdul\OneDrive\Desktop\TASK 04
ERNSHIP/task5_linear_algebra.py"
Matrix A:
[[1 2]
 [3 4]]

Matrix B:
[[5 6]
 [7 8]]

Dot Product:
[[19 22]
 [43 50]]

Matrix Multiplication:
[[19 22]
 [43 50]]

Transpose of Matrix A:
[[1 3]
 [2 4]]

Inverse of Matrix A:
[[-2.   1. ]
 [ 1.5 -0.5]]
```

Conclusion

- Vectorized operations perform computations on entire arrays simultaneously, making them much faster and more efficient than traditional Python loops. They are preferred for numerical computing and data analysis.

Part B: Pandas Fundamentals

Aim

Create and manipulate Series and DataFrames, perform filtering, sorting, groupby, merge, and join.

Python File

pandas_fundamentals.py

Output Screenshot

```
PS C:\Users\abdul\OneDrive\Desktop\TASK 04 INTERN
ERNSHIP/pandas_fundamentals.py"
2    Sara   19         CS     78     Unpaid
3  Ayesha   22         IT     92       Paid

Joined DataFrame:
     Name  Age Department  Marks Fee_Status
0     Ali   20         CS     85       Paid
1   Ahmed   21         SE     90       Paid
2    Sara   19         CS     78     Unpaid
3  Ayesha   22         IT     92       Paid
```

Conclusion

Pandas makes data manipulation and analysis easy and efficient.

Part C: Mini Project

Aim

Analyze the Titanic dataset using Pandas.

Dataset Description

The Titanic dataset contains passenger information such as age, gender, ticket class, fare, and survival status. It is commonly used for learning data analysis and machine learning.

Data Cleaning

- Filled missing Age values with the mean.
- Filled missing Embarked values with the most frequent value.
- Replaced missing Cabin values with "Unknown".

Analysis Performed

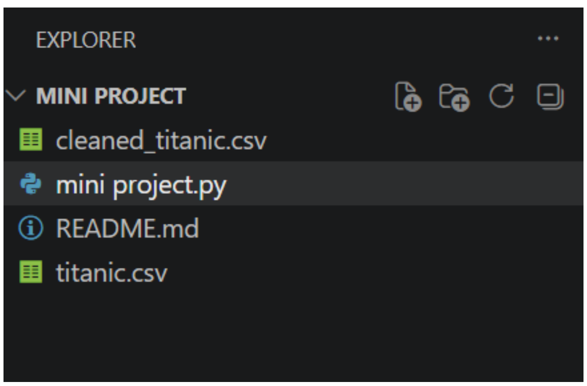
- Displayed first five rows.
- Checked missing values.
- Generated summary statistics.
- Calculated average age.
- Counted survived passengers.
- Average fare by passenger class.
- Survival rate by gender.

Python File

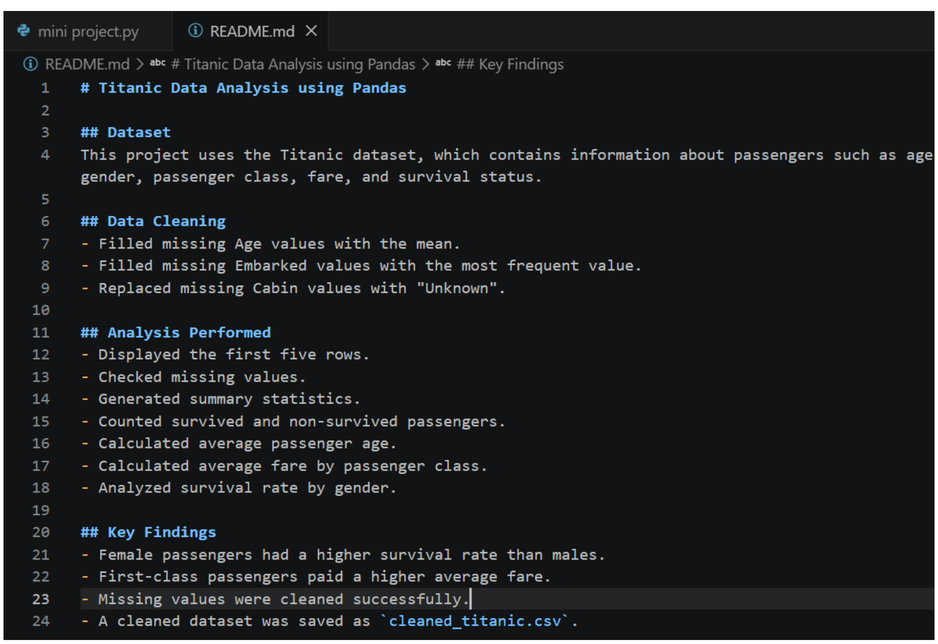
mini_project.py

Output Screenshot

CLEANED TITANIC CSV CREATED:



README FILE CREATED:



FINAL OUTPUT:

```
PS C:\Users\abdul\OneDrive\Desktop\TASK 04 INTERNSHIP\mini project>
p/TASK 04 INTERNSHIP/mini project/mini project.py"
Name: Fare, dtype: float64

Survival by Gender:
Sex
female    0.742038
male      0.188908
Name: Survived, dtype: float64

Cleaned dataset saved as cleaned_titanic.csv
```

Key Findings

- Female passengers had a higher survival rate.
- First-class passengers paid the highest average fare.
- Missing values were cleaned successfully.
- The cleaned dataset was saved successfully.

Project Conclusion

This project demonstrated the use of **Pandas** for data analysis using the Titanic dataset. Missing values were cleaned, summary statistics were generated, and basic exploratory data analysis was performed. The analysis showed useful insights such as survival rates by gender and average fares by passenger class, improving practical understanding of data cleaning and analysis in Python.

PROJECT PUSHED TO GITHUB: